UNIT IV: Inheritance & Pointers

Section A: Debugging Questions

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

Topic: Hybrid Inheritance, Virtual Base Classes, Ambiguity

Corrected Code:

```
#include <iostream>
using namespace std;

class Person {
public:
    int idNumber;

    Person(int id = 0) : idNumber(id) {
        cout << "Person initialized: " << idNumber << endl;
    }
};

class Student : virtual public Person {
public:
    Student() {
        cout << "Student initialized" << endl;
    }
};

class Employee : virtual public Person {
public:
    Employee() {
        cout << "Employee initialized" << endl;
    }
};

class TeachingAssistant : public Student, public Employee {
public:
```

```
    TeachingAssistant(int id) : Person(id), Student(), Employee() {
        cout << "Teaching Assistant initialized" << endl;
    }
};

int main() {
    TeachingAssistant ta(101);

    cout << "TA ID: " << ta.idNumber << endl;

    return 0;
}
```

Output:

```
Person initialized: 101
Student initialized
Employee initialized
Teaching Assistant initialized
TA ID: 101
```

Question 2: Missing Base Constructor in Derived Initializer List

Topic: Constructors in Derived Classes, Order of Constructor Execution

Corrected Code:

```
#include <iostream>
#include <string>
using namespace std;

class Device {
protected:
    string macAddress;

public:
    Device(string mac) : macAddress(mac) {
        cout << "Device configured with MAC: "
             << macAddress << endl;
    }
};
```

```
class Sensor : public Device {
private:
    int samplingRate;

public:
    Sensor(string mac, int rate)
        : Device(mac), samplingRate(rate) {
        cout << "Sensor initialized" << endl;
    }

    void display() const {
        cout << "MAC: " << macAddress
             << ", Rate: " << samplingRate
             << " Hz" << endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);

    tempSensor.display();

    return 0;
}
```

Output: Device configured with MAC: 00:1A:2B:3C:4D:5E
Sensor initialized
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz

Question 3: Base Class Pointer Member Access & Object Slicing

Topic: Pointers to Derived Classes, Static Type Checking, Object Slicing

Corrected Code:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    int maxSpeed;
```

```
    Vehicle(int s) : maxSpeed(s) {}

    virtual void honk() {
        cout << "General Horn" << endl;
    }

    virtual ~Vehicle() {}
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;

    SportsCar(int s, bool turbo)
        : Vehicle(s), turboEnabled(turbo) {}

    void engageTurbo() {
        cout << "Turbo Engaged!" << endl;
    }
};

int main() {
    Vehicle *vPtr = new SportsCar(300, true);

    vPtr->honk();

    SportsCar *sPtr = dynamic_cast<SportsCar*>(vPtr);

    if (sPtr != nullptr) {
        sPtr->engageTurbo();
    }

    cout << "Maximum Speed: " << sPtr->maxSpeed << endl;

    // Prevent slicing by using a reference.
```

```
    SportsCar car(250, true);
    Vehicle &vehicleRef = car;

    cout << "Car Speed: " << vehicleRef.maxSpeed << endl;

    delete vPtr;

    return 0;
}
```

Output: General Horn
Turbo Engaged!
Maximum Speed: 300
Car Speed: 250

Question 4: Incorrect Return Type in `this` Pointer Method Chaining

Topic: The `this` Pointer, Method Chaining (Fluent Interface)

Corrected Code:

```
#include <iostream>
using namespace std;

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
```

```
    }

    void display() const {
        cout << "Window: " << width
             << " x " << height << endl;
    }
};

int main() {
    WindowBox box;

    box.setWidth(800).setHeight(600);

    box.display();

    return 0;
}
```

Output:

```
Window: 800 x 600
```

Question 5: Private Inheritance Access Violations & Inaccessible Base Cast

Topic: Access Specifiers in Inheritance (`private` derivation), Upcasting

Corrected Code:

```
#include <iostream>
using namespace std;

class RawSocket {
public:
    void openPort(int port) {
        cout << "Port " << port << " opened." << endl;
    }

    void closePort() {
        cout << "Port closed." << endl;
    }
};
```

```
class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);
        cout << "Tunnel encrypted with TLS 1.3" << endl;
    }
};

int main() {
    SecureChannel channel;

    channel.establishEncryptedTunnel(443);
    channel.closePort();

    return 0;
}
```

Output:

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

Section B: Pseudo-Code Writing Questions

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Topic: Hybrid Inheritance, Virtual Base Classes, Diamond Resolution

pseudo-code:

```
Class 1: Person (Root Base Class)
CLASS Person
    DECLARE personID AS INTEGER
    DECLARE fullName AS STRING
    DECLARE email AS STRING
    CONSTRUCTOR Person(id, name, mail)
        personID <- id
```

```
        fullName <- name
        email <- mail
    END CONSTRUCTOR
END CLASS
```

Class 2: Student (Virtual Inheritance)

```
CLASS Student INHERITS VIRTUAL Person
    DECLARE gpa AS REAL
    DECLARE creditsEarned AS INTEGER
    CONSTRUCTOR Student(g, credits)
        gpa <- g
        creditsEarned <- credits
    END CONSTRUCTOR
END CLASS
```

Class 3: Faculty (Virtual Inheritance)

```
CLASS Faculty INHERITS VIRTUAL Person
    DECLARE academicDepartment AS STRING
    DECLARE rankTitle AS STRING
    CONSTRUCTOR Faculty(dept, rank)
        academicDepartment <- dept
        rankTitle <- rank
    END CONSTRUCTOR
END CLASS
```

Class 4: TeachingAssistant (Most Derived Class)

```
CLASS TeachingAssistant INHERITS Student, Faculty
    DECLARE stipendAmount AS REAL
    DECLARE assignedCourse AS STRING
    CONSTRUCTOR TeachingAssistant(id, name, email,
                                  g, credits, dept, rank,
                                  stipend, course)
        BASE Person(id, name, email)
        BASE Student(g, credits)
        BASE Faculty(dept, rank)
        stipendAmount <- stipend
        assignedCourse <- course
    END CONSTRUCTOR
    METHOD DisplayTARecord()
        DISPLAY "Person ID: ", personID
        DISPLAY "Name: ", fullName
```

output:

```
----- TEACHING ASSISTANT RECORD -----
Person ID: 501
Name: Arun Kumar
```

```
GPA: 8.7
Department: Computer Science
Stipend: 15000
Assigned Course: C++ Programming
```

Question 2: Polymorphic Payroll with Abstract Base Class & Base Pointers

Topic: Abstract Classes, Pure Virtual Functions, Pointers to Derived Classes

pseudo-code:

1. Structured pseudocode

```
Abstract Base Class: Employee
ABSTRACT CLASS Employee
    DECLARE employeeID AS STRING
    DECLARE name AS STRING
    CONSTRUCTOR Employee(id, empName)
        employeeID <- id
        name <- empName
    END CONSTRUCTOR
    ABSTRACT METHOD CalculatePay() RETURNS REAL
    METHOD DisplayHeader()
        DISPLAY "Employee ID: ", employeeID
        DISPLAY "Employee Name: ", name
    END METHOD
END CLASS
```

Derived Class: SalariedEmployee

```
CLASS SalariedEmployee EXTENDS Employee
    DECLARE monthlySalary AS REAL
    CONSTRUCTOR SalariedEmployee(id, name, salary)
        BASE Employee(id, name)
        monthlySalary <- salary
    END CONSTRUCTOR
    METHOD CalculatePay() RETURNS REAL
        RETURN monthlySalary
    END METHOD
END CLASS
```

Derived Class: HourlyEmployee

```
CLASS HourlyEmployee EXTENDS Employee
    DECLARE hourlyWage AS REAL
    DECLARE hoursWorked AS INTEGER
    CONSTRUCTOR HourlyEmployee(id, name, wage, hours)
        BASE Employee(id, name)
        hourlyWage <- wage
        hoursWorked <- hours
```

```
    END CONSTRUCTOR
    METHOD CalculatePay() RETURNS REAL
        IF hoursWorked <= 40 THEN
            RETURN hourlyWage * hoursWorked
        ELSE
            regularPay <- 40 * hourlyWage
            overtimeHours <- hoursWorked - 40
            overtimePay <- overtimeHours * hourlyWage * 1.5
            RETURN regularPay + overtimePay
        END IF
    END METHOD
END CLASS
```

Main Program: Batch Processor

```
BEGIN
    DECLARE employees AS ARRAY OF POINTER TO Employee
    DECLARE totalPay AS REAL <- 0
    employees[0] <- NEW SalariedEmployee(
        "E101", "Arun", 8500)
    employees[1] <- NEW HourlyEmployee(
        "E102", "Bala", 50, 48)
    FOR i <- 0 TO 1
        employees[i]->DisplayHeader()
        pay <- employees[i]->CalculatePay()
        DISPLAY "Salary: ", pay
        totalPay <- totalPay + pay
    END FOR
```

output:

```
Employee ID: E101
Employee Name: Arun
Salary: 8500
Employee ID: E102
Employee Name: Bala
Salary: 2600
Total Payroll: 11100
```

Question 3: Composite Canvas Engine with Member Classes & `this` Chaining

Topic: Member Classes (Composition), `this` Pointer Chaining

pseudo-code:

1. Structured pseudocode

Class: Point2D

```
CLASS Point2D
```

```
    DECLARE x AS REAL
    DECLARE y AS REAL
    CONSTRUCTOR Point2D(xValue = 0.0, yValue = 0.0)
        x <- xValue
        y <- yValue
    END CONSTRUCTOR
END CLASS
```

Class: Circle

```
CLASS Circle
    DECLARE centerPoint AS Point2D
    DECLARE radius AS REAL
    CONSTRUCTOR Circle()
        centerPoint <- NEW Point2D(0.0, 0.0)
        radius <- 1.0
    END CONSTRUCTOR
    METHOD SetCenter(newX, newY) RETURNS REF Circle
        centerPoint.x <- newX
        centerPoint.y <- newY
        RETURN REF THIS
    END METHOD
    METHOD SetRadius(r) RETURNS REF Circle
        IF r > 0 THEN
            radius <- r
        ELSE
            DISPLAY "Invalid radius"
        END IF
        RETURN REF THIS
    END METHOD
    METHOD Draw()
        DISPLAY "Circle Specifications"
        DISPLAY "Center: (", centerPoint.x, ", ",
                centerPoint.y, ")"
        DISPLAY "Radius: ", radius
    END METHOD
END CLASS
```

Main Program

```
BEGIN
    DECLARE myCircle AS Circle
    myCircle.SetCenter(12.5, 30.0)
            .SetRadius(7.5)
            .Draw()
END
```

output:

```
Circle Specifications
Center: (12.5, 30)
Radius: 7.5
```

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

Topic: Multilevel Inheritance, Constructors in Derived Classes, Base Pointers

pseudo-code:

```
Class 1: Vehicle (Base Class)
CLASS Vehicle
    DECLARE vehicleVIN AS STRING
    DECLARE baseWeightKg AS REAL
    CONSTRUCTOR Vehicle(vin, weight)
        vehicleVIN <- vin
        baseWeightKg <- weight
    END CONSTRUCTOR
    METHOD DisplaySpecs()
        DISPLAY "Vehicle VIN: ", vehicleVIN
        DISPLAY "Base Weight: ", baseWeightKg, " kg"
    END METHOD
END CLASS
Class 2: AutonomousVehicle
CLASS AutonomousVehicle EXTENDS Vehicle
    DECLARE lidarSensorID AS STRING
    DECLARE firmwareVersion AS STRING
    CONSTRUCTOR AutonomousVehicle(vin, weight, lidar, firmware)
        BASE Vehicle(vin, weight)
        lidarSensorID <- lidar
        firmwareVersion <- firmware
    END CONSTRUCTOR
END CLASS
Class 3: DeliveryDrone
CLASS DeliveryDrone EXTENDS AutonomousVehicle
    DECLARE payloadCapacityKg AS REAL
    DECLARE batteryChargePercent AS REAL
    CONSTRUCTOR DeliveryDrone(vin, weight, lidar, firmware,
                              payload, battery)
        BASE AutonomousVehicle(vin, weight, lidar, firmware)
        payloadCapacityKg <- payload
        batteryChargePercent <- battery
    END CONSTRUCTOR
    METHOD DisplayFlightStatus()
```

```
        CALL DisplaySpecs()
        DISPLAY "LiDAR Sensor ID: ", lidarSensorID
        DISPLAY "Firmware Version: ", firmwareVersion
        DISPLAY "Payload Capacity: ", payloadCapacityKg, " kg"
        DISPLAY "Battery Charge: ", batteryChargePercent, "%"
    END METHOD
END CLASS
```

Main Program

```
BEGIN
    DECLARE dronePtr AS POINTER TO DeliveryDrone
    DECLARE vehiclePtr AS POINTER TO Vehicle
    dronePtr <- NEW DeliveryDrone(
        "VIN1001", 12.5, "LIDAR-01",
        "V2.5", 5.0, 90.0)
    vehiclePtr <- dronePtr
    DISPLAY "General Vehicle Information"
    vehiclePtr->DisplaySpecs()
    DISPLAY "Complete Drone Information"
    DECLARE derivedPtr AS POINTER TO DeliveryDrone
    derivedPtr <- CAST vehiclePtr TO POINTER TO DeliveryDrone
    derivedPtr->DisplayFlightStatus()
    DELETE dronePtr
END
```

2. Constructor invocation order

```
The inheritance hierarchy is:
Vehicle
   |
   v
AutonomousVehicle
   |
   v
DeliveryDrone
```

output:

General Vehicle Information

Vehicle VIN: VIN1001

Base Weight: 12.5 kg

Complete Drone Information

Vehicle VIN: VIN1001

Base Weight: 12.5 kg

LiDAR Sensor ID: LIDAR-01

Firmware Version: V2.5

Payload Capacity: 5 kg

Battery Charge: 90%

Question 5: Dynamic Object Pointer Array & Zero-Copy Sorting Engine

Topic: Pointers to Objects, Arrow Operator (`->`), Pointer Manipulation

pseudo-code:

1. Structured pseudocode

```
Class: Product
CLASS Product
    DECLARE sku AS STRING
    DECLARE price AS REAL
    DECLARE stockCount AS INTEGER
    CONSTRUCTOR Product(code, cost, stock)
        sku <- code
        price <- cost
        stockCount <- stock
    END CONSTRUCTOR
    METHOD PrintProduct()
        DISPLAY "SKU: ", sku
        DISPLAY "Price: $", price
        DISPLAY "Stock: ", stockCount
    END METHOD
END CLASS
Pointer-swap Bubble Sort
BEGIN
    DECLARE N <- 3
    DECLARE catalog AS ARRAY OF POINTER TO Product
    catalog[0] <- NEW Product("Laptop", 1200, 10)
    catalog[1] <- NEW Product("Mouse", 25, 50)
    catalog[2] <- NEW Product("Monitor", 350, 20)
    DISPLAY "Before Sorting"
    FOR i <- 0 TO N-1
        catalog[i]->PrintProduct()
    END FOR
    FOR i <- 0 TO N-2
        FOR j <- 0 TO N-i-2
            IF catalog[j]->price > catalog[j+1]->price THEN
                temp <- catalog[j]
                catalog[j] <- catalog[j+1]
                catalog[j+1] <- temp
            END IF
        END FOR
```

```
    END FOR
    DISPLAY "After Sorting"
    FOR i <- 0 TO N-1
        catalog[i]->PrintProduct()
    END FOR
    FOR i <- 0 TO N-1
        DELETE catalog[i]
    END FOR
END
```

output:

```
Before Sorting

SKU: Laptop

Price: $1200

Stock: 10

SKU: Mouse

Price: $25

Stock: 50

SKU: Monitor

Price: $350

Stock: 20

After Sorting

SKU: Mouse

Price: $25

Stock: 50

SKU: Monitor

Price: $350

Stock: 20

SKU: Laptop

Price: $1200

Stock: 10
```